

3.a) Group Operations:

Question:

Implement $\text{SE}_2(2)$ compose and inverse

Solution:

Code snippet 3 shows the implementation of `se22_compose(X1, X2)` and `se22_inverse(X)`. Code snippet 4 shows its unit test and Fig 1 shows its passing.

3.b) Lie Algebra $\mathfrak{se}_2(2)$:

Question:

Implement `se22_wedge(xi)` and `se22_vee(Xi)`

Solution:

Code snippet 3 shows the implementation of `se22_wedge(xi)` and `se22_vee(Xi)`. Code snippet 4 shows its unit test and Fig 1 shows its passing.

3.c) Exponential and Logarithm Maps:

Question:

Implement `se22_exp(xi)` and `se22_log(X)`. Verify round-trip: $\text{Exp}(\text{Log}(X)) = X$ for at least 3 random $X \in \text{SE}_2(2)$ (i.e., `se22_exp(se22_log(X))` recovers X to machine precision).

Solution:

Code snippet 3 shows the implementation of `se22_exp(xi)` and `se22_log(X)`.

Code snippet 4 shows its unit test and Fig 1 shows its passing.

3.d) Adjoint Ad_X and Small Adjoint ad_ξ :

Question:

Implement `se22_Ad(X)` and `se22_ad(xi)`. Verify numerically (for at least 3 random inputs each):

- $\text{Ad}_{X_1 X_2} = \text{Ad}_{X_1} \text{Ad}_{X_2}$
- $[\xi_1, \xi_2] = \text{ad}_{\xi_1} \xi_2$

Solution:

Code snippet 3 shows the implementation of the functions `se22_Ad(X)` and `se22_ad(xi)`. Code snippet 4 shows the implementation of the unit test verifying $\text{Ad}_{X_1 X_2} = \text{Ad}_{X_1} \text{Ad}_{X_2}$ for random $X_1, X_2 \in \text{SE}_2(2)$ for random $X \in \text{SE}_2(2)$ named `test_se22_Ad_composition` and Fig 1 shows the test passing. Code snippet 4 shows the implementation of the unit test verifying $[\xi_1, \xi_2] = \text{ad}_{\xi_1} \xi_2$ for random $\xi_1, \xi_2 \in \mathfrak{se}_2(2)$ named `test_se22_ad` and Fig 1 shows the test passing.

Implementation and Unit Tests

Code Snippet 1: $\text{SE}_2(2)$ and $\mathfrak{se}_2(2)$ Function Implementations

```

1 import numpy as np
2 from Code.S02.S02_maps import so2_wedge, so2_vee, so2_exp, so2_log
3
4 def se22_compose(X1, X2):
5     """
6     Composition of elements of SE2(2)
7
8     :param X1: element of SE2(2)
9     :param X2: element of SE2(2)
10    """
11    X1_t = X1[0:2, 3].reshape((2, 1))
12    X1_v = X1[0:2, 2].reshape((2, 1))
13    X1_R = X1[0:2, 0:2]
14
```

```

15     X2_t = X2[0:2, 3].reshape((2, 1))
16     X2_v = X2[0:2, 2].reshape((2, 1))
17     X2_R = X2[0:2, 0:2]
18
19     X12_R = X1_R @ X2_R
20     X12_v = X1_R @ X2_v + X1_v
21     X12_t = X1_R @ X2_t + X1_t
22
23     X12 = np.block([[X12_R, X12_v, X12_t],
24                     [np.zeros((2, 2)), np.eye(2)]]))
25
26     return X12
27
28 def se22_inverse(X):
29     """
30     Inverse of SE2(2) element
31
32     :param X: element of SE2(2)
33     """
34     X_t = X[0:2, 3].reshape((2, 1))
35     X_v = X[0:2, 2].reshape((2, 1))
36     X_R = X[0:2, 0:2]
37
38     X_inv_t = -X_R.T @ X_t
39     X_inv_v = -X_R.T @ X_v
40     X_inv_R = X_R.T
41
42     X_inv = np.block([[X_inv_R, X_inv_v, X_inv_t],
43                     [np.zeros((2, 2)), np.eye(2)]]))
44
45     return X_inv
46
47 def se22_wedge(xi):
48     """
49     Maps from se22 real number parametrization to se22 Lie algebra
50
51     :param xi: twist vector (a_x, a_y, b_x, b_y, omega) in R3
52     """
53     xiwedge = np.block([[so2_wedge(xi[4]), xi[0:2].reshape((2, 1)), xi[2:4].reshape((2, 1))
54 ],
55                     [np.zeros((2, 2)), np.zeros((2, 2))]])
56
57     return xiwedge
58
59 def se22_vee(xiwedge):
60     """
61     Maps from se22 Lie algebra to its real number parametrization
62
63     :param xiwedge: 3x3 se22 Lie algebra element
64     """
65     xi = np.block([xiwedge[[0, 1], [2, 2]], xiwedge[[0, 1], [3, 3]], so2_vee(xiwedge[0:2,
66 0:2])]);
67
68     return xi
69
70 def se22_exp(xi):
71     """
72     Maps from parametrization of se22 Lie algebra to SE22 Lie group
73
74     :param xi: twist vector (a_x, a_y, b_x, b_y, omega) in R3
75     """
76     # Extract elements
77     a = xi[0:2]
78     b = xi[2:4]
79     omega = xi[4]
80     # S02 exponential map

```

```

81 R = so2_exp(omega)
82
83 # Translation-orientation coupling
84 if abs(omega) > 1e-8:
85     V = np.sin(omega) / omega * np.eye(2) + (1 - np.cos(omega)) / omega * np.array([[0,
86 -1], [1, 0]])
87 else:
88     V = ((1 - omega ** 2 / 6 + omega ** 4 / 120) * np.eye(2)
89         + (omega / 2 - omega ** 3 / 24) * np.array([[0, -1], [1, 0]]))
90
91 X = np.block([[R, V @ a.reshape((2, 1)), V @ b.reshape((2, 1))],
92               [np.zeros((2, 2)), np.eye(2)]])
93
94 return X
95
96 def se22_log(xiwedge):
97     """
98     Maps from SE22 Lie group to the parametrization of its Lie algebra se22
99
100     :param xiwedge: 3x3 se22 Lie algebra element
101     """
102     omega = so2_log(xiwedge[0:2, 0:2])
103
104     if abs(omega) > 1e-8:
105         V_inv = omega / 2 * (np.cos(omega / 2) / np.sin(omega / 2) * np.eye(2) + np.array
106             ([[0, 1], [-1, 0]]))
107     else:
108         V_inv = 1 / 2 * ((2 - omega ** 2 / 6 - omega ** 4 / 360 - 2 * omega ** 6 / 30240 +
109             omega ** 8 / 604800) * np.eye(2) + omega * np.array([[0, -1], [1, 0]]))
110
111     a = V_inv @ xiwedge[0:2, 2]
112     b = V_inv @ xiwedge[0:2, 3]
113
114     xi = np.vstack((a.reshape((2, 1)), b.reshape((2, 1)), np.reshape(omega, (1,1))))
115
116 return xi
117
118 def se22_Ad(X):
119     """
120     Maps from se22 to se22 defined by (Ad_X xi)**wedge = X xi**wedge X**-1
121     Shifts tangent spaces
122
123     :param X: SE22 Lie group element
124     """
125     X_t = X[0:2, 3].reshape((2, 1))
126     X_v = X[0:2, 2].reshape((2, 1))
127     X_R = X[0:2, 0:2]
128
129     J = np.array([[0, -1], [1, 0]])
130     v_circ = -J @ X_v
131     t_circ = -J @ X_t
132
133     Ad = np.block([[X_R, np.zeros((2, 2)), v_circ],
134                   [np.zeros((2, 2)), X_R, t_circ],
135                   [np.zeros((1, 4)), 1]])
136
137 return Ad
138
139 def se22_ad(xi):
140     """
141     Derivative of Adjoint at identity
142
143     :param xi: se2(2) Lie algebra element parametrization
144     """
145     a = xi[0:2].reshape((2, 1))
146     b = xi[2:4].reshape((2, 1))
147     omega = xi[4]

```

```

146 J = np.array([[0, -1], [1, 0]])
147 a_circ = -J @ a
148 b_circ = -J @ b
149
150 omega_wedge = so2_wedge(omega)
151
152 ad = np.block([[omega_wedge, np.zeros((2, 2)), a_circ],
153               [np.zeros((2, 2)), omega_wedge, b_circ],
154               [np.zeros((1, 4)),
155               0]])
156 return ad

```

Code Snippet 2: Unit Tests for $SE_2(2)$ Using Pytest

```

1 import pytest as pt
2 import numpy as np
3 from Code.SE22.SE22_maps import se22_compose, se22_inverse, se22_exp, se22_log, se22_vee,
   se22_wedge, se22_Ad, se22_ad
4
5 def test_se22_exp_log():
6     """
7     Test that exp(log(X)) = X for random SE22 elements
8     """
9     n_samples = 100
10
11     rng = np.random.default_rng()
12     xi_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
13
14     checks = np.zeros([n_samples, 1])
15     for i in range(n_samples):
16         X_random = se22_exp(xi_random[:, i])
17
18         X_log = se22_log(X_random)
19         X_tilde = se22_exp(X_log)
20
21         checks[i] = np.all(X_tilde == pt.approx(X_random))
22
23     assert np.all(checks)
24
25 def test_se22_Ad():
26     """
27     Test that Ad_X(xi) = (X xi^wedge X^-1)^vee for random SE22, se22 elements
28     """
29     n_samples = 100
30
31     rng = np.random.default_rng()
32     xi_X_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
33     xi_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
34
35     checks = np.zeros([n_samples, 1])
36     for i in range(n_samples):
37         X_random = se22_exp(xi_X_random[:, i])
38
39         xi_prime = se22_vee(X_random @ se22_wedge(xi_random[:, i]) @ se22_inverse(X_random))
40         xi_prime_ad = se22_Ad(X_random) @ xi_random[:, i]
41
42         checks[i] = np.all(xi_prime_ad == pt.approx(xi_prime))
43
44     assert np.all(checks)
45
46 def test_se22_Ad_composition():
47     """
48     Test that Ad_{X_1 X_2} = Ad_{X_1}Ad_{X_2} for random SE22 elements
49     """
50     n_samples = 100
51
52     rng = np.random.default_rng()
53     xi1_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))

```

```

54     xi2_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
55
56     checks = np.zeros([n_samples, 1])
57     for i in range(n_samples):
58         X1_random = se22_exp(xi1_random[:, i])
59         X2_random = se22_exp(xi2_random[:, i])
60
61         Ad_X1X2 = se22_Ad(se22_compose(X1_random, X2_random))
62         Ad_X1_Ad_X2 = se22_Ad(X1_random) @ se22_Ad(X2_random)
63
64         checks[i] = np.all(Ad_X1X2 == pt.approx(Ad_X1_Ad_X2))
65
66     assert np.all(checks)
67
68 def test_se22_Ad_inv():
69     """
70     Test that  $(\text{Ad}_X)^{-1} = \text{Ad}_{\{X^{-1}\}}$  for random SE22 elements
71     """
72     n_samples = 100
73
74     rng = np.random.default_rng()
75     xi_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
76
77     checks = np.zeros([n_samples, 1])
78     for i in range(n_samples):
79         X_random = se22_exp(xi_random[:, i])
80
81         Ad_X_inv = np.linalg.inv(se22_Ad(X_random))
82         Ad_Xinv = se22_Ad(se22_inverse(X_random))
83
84         a = se22_Ad(X_random) @ Ad_X_inv
85         b = se22_Ad(X_random) @ Ad_Xinv
86
87         checks[i] = np.all(Ad_X_inv == pt.approx(Ad_Xinv))
88
89     assert np.all(checks)
90
91 def test_se22_ad():
92     """
93     Test that  $\text{ad}_{\{xi_1\}}(xi_2) = [xi_1, xi_2]$  for random se22 elements
94     """
95     n_samples = 100
96
97     rng = np.random.default_rng()
98     xi1_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
99     xi2_random = np.reshape(rng.uniform(-10, 10, 5 * n_samples), (5, n_samples))
100
101     checks = np.zeros([n_samples, 1])
102     for i in range(n_samples):
103         Lie_bracket = se22_vee(se22_wedge(xi1_random[:, i]) @ se22_wedge(xi2_random[:, i])
104                                - se22_wedge(xi2_random[:, i]) @ se22_wedge(xi1_random[:, i]))
105         Lie_bracket_ad = se22_ad(xi1_random[:, i]) @ xi2_random[:, i]
106
107         checks[i] = np.all(Lie_bracket_ad == pt.approx(Lie_bracket))
108
109     assert np.all(checks)

```

Figure 1: Pytest unit test output showing tests passing

```
(.venv) PS pytest Tests\SE22\test_SE22.py          Classes\AAE 590LGM\AAE590LGM-Lie-Group-Methods>
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\thatf\OneDrive\Documents\Purdue Classes\AAE 590LGM\AAE590LGM-Lie-Group-Methods
collected 5 items

Tests\SE22\test_SE22.py ..... [100%]

===== 5 passed in 0.28s =====
```